INF560 / Unidad 3 / GL08

GUÍA DE LABORATORIO

Relaciones Eloquent

Categorías y etiquetas: 1:N y N:M, claves foráneas y eager loading en eventos-app

Laravel 13 · Eloquent · Relaciones · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 4 h

Métodos HTTP de la serie: GET POST PUT DELETE

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

Hasta ahora eventos-app tiene una sola tabla de negocio: `eventos`. En esta guía convertimos las categorías en una **tabla real** y la **conectamos** con eventos: cada evento **pertenecerá a una categoría** (relación uno a muchos) y, además, podrá llevar varias **etiquetas** (relación muchos a muchos). Estas relaciones son el corazón de una base de datos relacional.

Objetivos de la guía

- Crear el modelo `Categoria` con su migración y seeder, reemplazando la lista fija de categorías.
- Agregar la clave foránea `categoria_id` a `eventos` con una migración.
- Definir la relación 1:N con `belongsTo` y `hasMany`.
- Usar la relación en el formulario, la tarjeta `x-evento-card`, el detalle y el filtro de la cartelera.
- Entender el problema `N+1` y resolverlo con `with()` y `load()` (eager loading).
- Crear la relación N:M `eventos` ↔ `etiquetas` con tabla pivote y `belongsToMany`.

Requisitos previos

- La GL07 completa: CRUD de eventos con `Route::resource('eventos')`, validación en el controlador y el formulario compartido `partials/evento-form.blade.php`.
- Base de datos PostgreSQL configurada en `.env` (`eventos_app`) y las migraciones al día.

Punto de partida del proyecto

La guía parte del estado actual del repositorio del curso (commit «Agrega CRUD eventos»). Esto es lo que ya existe y lo que cambiará:

Elemento	Estado actual	En esta guía
Tabla `eventos`	titulo, slug, descripcion, fecha, lugar, cupo, precio, imagen, publicado, destacado	+ `categoria_id` (FK)
Categorías	Texto fijo en `Badge.php` y en `partials/filtros`	Tabla `categorias` + modelo
`EventoController`	CRUD con `$request->validate()` en store y update	Pasa categorías/etiquetas, valida, `sync()`
`index()`	`paginate(3)` sin relaciones	`with('categoria')` + filtro real
`partials/filtros`	Select sin `name`: el filtro no se envía	Filtra por categoría y destacados
`x-evento-card`	Sin información de categoría	Muestra `x-badge` de la categoría
Seeders	`EventolSeeder` (20 eventos)	+ `CategoriaSeeder`, `EtiquetaSeeder`

> **ALCANCE DE ESTA GUÍA**
> Trabajamos las dos relaciones más comunes: **1:N** (categoría ↔ eventos) y **N:M** (eventos ↔ etiquetas). Otras relaciones (un organizador, inscripciones de usuarios) se apoyan en estos mismos patrones: la del organizador queda como trabajo independiente.

01 Teoría breve: relaciones y claves foráneas

1.1 Los tipos de relación

- **Uno a muchos (1:N):** una categoría tiene muchos eventos; cada evento pertenece a una categoría.
- **Muchos a muchos (N:M):** un evento tiene muchas etiquetas y una etiqueta pertenece a muchos eventos. Necesita una tabla intermedia (pivote).

1.2 La clave foránea

> DEFINICIÓN — CLAVE FORÁNEA (FOREIGN KEY)
>
> Una **clave foránea** es una columna que guarda el `id` de una fila en otra tabla. En `eventos.categoria_id` almacenamos a qué categoría pertenece cada evento. La base de datos **garantiza la integridad**: no permite un `categoria_id` que no exista en `categorias`.

1.3 Cómo lo modela Eloquent

En Eloquent, cada relación es un **método** en el modelo. Al declararlo, puede navegar la relación como si fuera una propiedad (`$evento->categoria`). Estas son las relaciones que construiremos:

Relación	Modelo	Método	Uso en código
Evento → 1 Categoría	Evento	belongsTo	$evento->categoria
Categoría → N Eventos	Categoria	hasMany	$categoria->eventos
Evento ↔ N Etiquetas	Evento	belongsToMany	$evento->etiquetas
Etiqueta ↔ N Eventos	Etiqueta	belongsToMany	$etiqueta->eventos

02 Ejercicio 1 — El modelo Categoria y la clave foránea

Antes de poder relacionar, la tabla del otro lado tiene que existir. Primero creamos `categorias`; después agregamos la columna que apunta a ella.

Paso 1 — Crear el modelo con migración y seeder

terminal Copiar

```
$ php artisan make:model Categoria -ms     # -m migración, -s seeder
```

database/migrations/XXXX_create_categorias_table.php Copiar

```
public function up(): void
{
    Schema::create('categorias', function (Blueprint $table) {
        $table->id();
        $table->string('nombre');
        $table->string('slug')->unique();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('categorias');
}
```

En el modelo, siga la misma convención que ya usa `Evento` (tabla explícita y `$fillable`):

app/Models/Categoria.php Copiar

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Categoria extends Model
{
    protected $table = 'categorias';

    protected $fillable = ['nombre', 'slug'];
}
```

Las categorías no son datos aleatorios: son un **catálogo fijo**. Por eso el seeder crea exactamente las mismas cuatro que hoy reconoce el componente `x-badge`:

database/seeders/CategoriaSeeder.php Copiar

```
use App\Models\Categoria;
use Illuminate\Support\Str;

public function run(): void
{
    // Mismos nombres que el match() de app/View/Components/Badge.php
    foreach (['Tecnologia', 'Cultura', 'Deporte', 'Finanzas'] as $nombre) {
        Categoria::create([
            'nombre' => $nombre,
            'slug'   => Str::slug($nombre),
        ]);
    }
}
```

> **NOTA**
> Si escribe «Tecnología» con tilde, el `match` de `Badge.php` no lo reconocerá y la insignia saldrá gris (rama `default`). Mantenga los nombres idénticos o actualice ambos lugares a la vez.

Paso 2 — Migración para agregar la clave foránea

terminal Copiar

```
$ php artisan make:migration add_categoria_id_to_eventos_table
```

database/migrations/XXXX_add_categoria_id_to_eventos_table.php Copiar

```
public function up(): void
{
    Schema::table('eventos', function (Blueprint $table) {
        $table->foreignId('categoria_id')
              ->nullable()
              ->constrained()      // referencia a la tabla categorias
              ->nullOnDelete();    // si se borra la categoría, queda en null
    });
}

public function down(): void
{
    Schema::table('eventos', function (Blueprint $table) {
        $table->dropConstrainedForeignId('categoria_id');
    });
}
```

Tres detalles del esquema:

- `foreignId('categoria_id')` crea una columna `bigint` sin signo y prepara la clave foránea.
- `constrained()` deduce la tabla `categorias` a partir del nombre de la columna (puede escribir `constrained('categorias')` para ser explícito).
- `nullable()` + `nullOnDelete()`: los eventos que ya existen no tienen categoría, y si se elimina una categoría usada sus eventos quedan sin categoría en vez de romperse.

> **EL ORDEN DE LAS MIGRACIONES IMPORTA**
>
> Laravel ejecuta las migraciones por la **fecha de su nombre**. La de `create_categorias_table` debe ser anterior a `add_categoria_id_to_eventos_table`; si no, PostgreSQL responde que la tabla `categorias` no existe. Si creó el modelo primero (Paso 1) el orden ya es correcto.
>
> Además, `->after('columna')` —que usan las migraciones de `publicado` y `destacado`— solo tiene efecto en MySQL/MariaDB; en PostgreSQL la columna nueva siempre se agrega al final. Por eso aquí no lo usamos.

Paso 3 — Asignar categoría en el factory y ordenar los seeders

Para que los eventos sembrados tengan categoría, el factory toma una al azar de las existentes:

database/factories/EventoFactory.php Copiar

```
use App\Models\Categoria;

// dentro de definition(), agregue al arreglo:
'categoria_id' => Categoria::inRandomOrder()->value('id'),
```

database/seeders/DatabaseSeeder.php Copiar

```php
$this->call([
    CategoriaSeeder::class,   // primero: deben existir las categorías
    EventolSeeder::class,     // el seeder de eventos del proyecto
]);
```

terminal Copiar

```
$ php artisan migrate:fresh --seed     # recrea las tablas y las puebla
```

> **ADVERTENCIA**
>
> `migrate:fresh` **borra todas las tablas** y los eventos que haya creado a mano desde el formulario. En desarrollo es lo habitual; nunca lo ejecute contra una base de datos de producción.

> **NOTA**
>
> El seeder de eventos del proyecto se llama `EventolSeeder` (con «l»). La guía respeta ese nombre para no romper nada; si decide renombrarlo a `EventoSeeder`, cambie también el nombre de la clase y la llamada en `DatabaseSeeder`.

03 Ejercicio 2 — Definir la relación 1:N

Una relación se declara en **ambos modelos**: el lado que «pertenece a» y el lado que «tiene muchos». Declarar el tipo de retorno (`BelongsTo`, `HasMany`) no es obligatorio, pero documenta el código y ayuda al editor.

En Evento — pertenece a una categoría

app/Models/Evento.php Copiar

```php
use Illuminate\Database\Eloquent\Relations\BelongsTo;

protected $fillable = [
    'titulo', 'slug', 'descripcion', 'fecha', 'lugar', 'cupo',
    'precio', 'imagen', 'publicado', 'destacado',
    'categoria_id',                      // nuevo: asignable desde el formulario
];

public function categoria(): BelongsTo
{
    return $this->belongsTo(Categoria::class);
}
```

> **NO OLVIDE EL $FILLABLE**
>
> Si `categoria_id` no está en `$fillable`, `Evento::create($datos)` lo **descarta en silencio**: la validación pasa, pero el evento se guarda sin categoría. Los factories no sufren este problema porque Laravel desactiva la protección al sembrar, por eso el error suele aparecer recién al usar el formulario.

En Categoria — tiene muchos eventos

app/Models/Categoria.php Copiar

```
use Illuminate\Database\Eloquent\Relations\HasMany;

public function eventos(): HasMany
{
    return $this->hasMany(Evento::class);
}
```

Probar la relación en Tinker

php artisan tinker Copiar

```
> $evento = Evento::first();
> $evento->categoria->nombre          // navega hacia la categoría
= "Deporte"
> $categoria = Categoria::where('slug', 'tecnologia')->first();
> $categoria->eventos->count()        // navega hacia sus eventos
= 6
// consulta encadenada sobre la relación
> $categoria->eventos()->where('publicado', true)->count();
// cuántos eventos tiene cada categoría
> Categoria::withCount('eventos')->get(['id', 'nombre']);
```

NOTA

Con paréntesis (`$categoria->eventos()`) obtiene el **constructor de consultas** para seguir filtrando; sin paréntesis (`$categoria->eventos`) obtiene ya la **colección** de resultados.

04 Ejercicio 3 — La relación en el formulario y las vistas

Paso 1 — Pasar las categorías a create() y edit()

El parcial `partials/evento-form` se usa en ambas vistas, así que las dos acciones deben entregarle la lista de categorías:

app/Http/Controllers/EventoController.php Copiar

```
use App\Models\Categoria;

public function create()
{
    $categorias = Categoria::orderBy('nombre')->get();
    return view('eventos.create', compact('categorias'));
}

public function edit(Evento $evento)
{
    $categorias = Categoria::orderBy('nombre')->get();
    return view('eventos.edit', compact('evento', 'categorias'));
}
```

Paso 2 — El selector en el formulario compartido

Agregue el campo dentro del bloque **Configuración** (el `aside`), justo antes de «Cupos». Reutiliza las variables `$campoNormal` y `$campoError` que el parcial ya define; no usamos `$campoBase` porque trae `pl-10` para el ícono.

resources/views/partials/evento-form.blade.php Copiar

```
<div>
    <label for="categoria_id" class="block text-sm font-medium text-slate-
700">Categoría</label>
    <select id="categoria_id" name="categoria_id"
        class="mt-1 w-full rounded-lg border bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 {{ $errors->has('categoria_id') ? $campoError : $campoNormal
}}">
        <option value="">— Selecciona una categoría —</option>
        @foreach ($categorias as $categoria)
            <option value="{{ $categoria->id }}"
                @selected(old('categoria_id', $evento?->categoria_id) == $categoria->id)>
                {{ $categoria->nombre }}
            </option>
        @endforeach
    </select>
    @error('categoria_id')
        <p class="mt-1.5 text-sm text-rose-600">{{ $message }}</p>
    @enderror
</div>
```

La directiva `@selected(...)` —que vio en la guía de Blade— marca la opción correcta al editar y conserva la elección con `old()` si la validación falla.

Paso 3 — Validar la categoría en store() y update()

Hoy las reglas están escritas dos veces, una en cada acción. Agregue la nueva regla en **ambos** arreglos de `$request->validate()`:

app/Http/Controllers/EventoController.php Copiar

```
$datos = $request->validate([
    'titulo'       => 'required|string|max:255',
    // ... reglas existentes ...
    // nueva: obligatoria y debe existir en la tabla categorias
    'categoria_id' => ['required', 'exists:categorias,id'],
]);
```

> **NOTA**
>
> Como `categoria_id` viene en `$datos` y ya está en `$fillable`, las líneas `Evento::create($datos)` y `$evento->update($datos)` lo guardan sin más cambios. La duplicación de reglas entre `store()` y `update()` es una buena razón para extraerlas a un **Form Request** más adelante.

Paso 4 — Mostrar la categoría en la tarjeta

El proyecto ya tiene el componente de clase `x-badge`, que recibe el nombre de una categoría y elige su color. Solo falta que `x-evento-card` lo use. Agregue la prop `categoria` y cambie la fila superior del cuerpo de la tarjeta:

resources/views/components/evento-card.blade.php Copiar

```
@props(['titulo', 'fecha', 'lugar', 'destacado', 'cupos',
        'descripcion' => null, 'imagen' => null, 'precio' => null,
        'publicado' => true, 'categoria' => null])

{{-- ... --}}

{{-- antes: class="mb-2 flex items-center justify-end" --}}
<div class="mb-2 flex items-center justify-between gap-2">
    @if ($categoria)
        <x-badge :categoria="$categoria" />
    @else
        <span class="text-xs text-slate-400">Sin categoría</span>
    @endif

    <span class="whitespace-nowrap rounded-full px-2.5 py-1 text-xs font-medium {{
$tiempoColor }}">
        {{ $tiempoRestante }}
    </span>
</div>
```

En la cartelera, pase el nombre de la categoría a la tarjeta (el resto de atributos no cambia):

resources/views/eventos/index.blade.php Copiar

```
<x-evento-card :titulo="$evento['titulo']" :fecha="$evento['fecha']" ...
    :categoria="$evento->categoria?->nombre">
```

> **NOTA**
>
> Usamos `$evento->categoria?->nombre` con el operador nullsafe `?->` porque los eventos creados antes de esta guía pueden no tener categoría (`null`); así la vista no se rompe.

Paso 5 — Mostrar la categoría en el detalle

En `show.blade.php`, dentro del contenedor de insignias sobre la portada (`absolute left-4 top-4`), agregue la categoría antes de «Destacado»:

resources/views/eventos/show.blade.php Copiar

```
<div class="absolute left-4 top-4 flex flex-wrap gap-2">
    @if ($evento->categoria)
        <x-badge :categoria="$evento->categoria->nombre" />
    @endif

    @if ($evento->destacado)
        {{-- ... insignia Destacado existente ... --}}
```

05 Ejercicio 4 — Eager loading, N+1 y el filtro de la cartelera

> **EL PROBLEMA N+1**
>
> La cartelera muestra 3 eventos por página y cada tarjeta accede a `$evento->categoria`. Eloquent hace **una consulta para los eventos** y luego **una por cada evento** para traer su categoría: 1 + 3 = 4 consultas. Con 20 eventos por página serían 21; con 100, 101. A esto se le llama problema **N+1**.

Paso 1 — La solución: with() en index()

Cargue la relación por adelantado. Laravel traerá todas las categorías de la página en **una sola consulta extra**:

app/Http/Controllers/EventoController.php Copiar

```
public function index()
{
    // Antes (1 + N consultas):
    //   Evento::orderByDesc('created_at')->latest()->paginate(3);
    $eventos = Evento::with('categoria')->latest()->paginate(3);

    return view('eventos.index', compact('eventos'));
}
```

Con `with('categoria')` las consultas pasan a ser **2**, sin importar cuántos eventos haya por página. De paso, sobraba una de las dos órdenes: `latest()` ya equivale a `orderByDesc('created_at')`.

Paso 2 — Que Laravel le avise del N+1

En desarrollo puede pedirle a Laravel que **lance una excepción** cada vez que una relación se cargue perezosamente dentro de un listado:

app/Providers/AppServiceProvider.php Copiar

```
use Illuminate\Database\Eloquent\Model;

public function boot(): void
{
    // Solo fuera de producción: detecta relaciones cargadas "tarde" (N+1)
    Model::preventLazyLoading(! $this->app->isProduction());
}
```

Pruébelo: quite temporalmente el `with('categoria')` y recargue la cartelera. Verá `LazyLoadingViolationException`. Vuelva a ponerlo y la página funciona de nuevo.

Paso 3 — load() en el detalle

En `show()` el evento ya llegó por route model binding; para precargar relaciones sobre un modelo existente se usa `load()` (eager loading «tardío»):

app/Http/Controllers/EventoController.php Copiar

```
public function show(Evento $evento)
{
    $evento->load('categoria');   // en el Ej. 5 se agrega 'etiquetas'
    return view('eventos.show', compact('evento'));
}
```

Paso 4 — Un filtro de cartelera que sí filtra

El parcial `partials/filtros.blade.php` tiene hoy dos detalles: el `select` no tiene `name` (el navegador nunca envía la categoría) y las opciones salen de un arreglo escrito a mano. Ahora que las categorías están en la base de datos, las tomamos de ahí y usamos la relación para filtrar:

resources/views/partials/filtros.blade.php Copiar

```
<select name="categoria" class="rounded-lg border-slate-300 text-sm">
    <option value="">Todas</option>
    @foreach ($categorias as $categoria)
        <option value="{{ $categoria->slug }}" @selected(request('categoria') === $categoria->slug)>
            {{ $categoria->nombre }}
        </option>
    @endforeach
</select>
```

app/Http/Controllers/EventoController.php Copiar

```
use Illuminate\Http\Request;

public function index(Request $request)
{
    $eventos = Evento::with('categoria')
        // filtra a través de la relación categoria
        ->when($request->input('categoria'), fn ($query, $slug) =>
            $query->whereRelation('categoria', 'slug', $slug))
        ->when($request->boolean('solo_destacados'), fn ($query) =>
            $query->where('destacado', true))
        ->latest()
        ->paginate(3)
        ->withQueryString();   // conserva el filtro al paginar

    $categorias = Categoria::orderBy('nombre')->get();

    return view('eventos.index', compact('eventos', 'categorias'));
}
```

- `when(valor, fn)` aplica la condición solo si el valor viene en la petición; si se elige «Todas» no filtra.
- `whereRelation('categoria', 'slug', $slug)` es la forma corta de `whereHas('categoria', fn ($q) => $q->where('slug', $slug))`.
- `withQueryString()` agrega `?categoria=...&solo_destacados=1` a los enlaces de paginación; sin él, al pasar de página se pierde el filtro.

> **NOTA**
> La casilla «Solo destacados» del parcial ya tenía `name`, pero nadie la leía. Con el segundo `when()` también empieza a funcionar. Pruebe `/eventos?categoria=deporte&solo_destacados=1` directamente en la barra de direcciones.

06 Ejercicio 5 — La relación N:M con etiquetas

Un evento puede tener varias etiquetas (Gratis, Aire libre, Taller) y una etiqueta aplica a muchos eventos. Esta relación N:M necesita una **tabla pivote** que guarde los pares (`etiqueta_id`, `evento_id`).

Paso 1 — Modelo Etiqueta y tabla pivote

terminal Copiar

```
$ php artisan make:model Etiqueta -ms
$ php artisan make:migration create_etiqueta_evento_table
```

database/migrations/XXXX_create_etiquetas_table.php Copiar

```
Schema::create('etiquetas', function (Blueprint $table) {
    $table->id();
    $table->string('nombre');
    $table->string('slug')->unique();
    $table->timestamps();
});
```

database/migrations/XXXX_create_etiqueta_evento_table.php Copiar

```
public function up(): void
{
    Schema::create('etiqueta_evento', function (Blueprint $table) {
        $table->foreignId('etiqueta_id')->constrained()->cascadeOnDelete();
        $table->foreignId('evento_id')->constrained()->cascadeOnDelete();
        $table->primary(['etiqueta_id', 'evento_id']);   // evita pares duplicados
    });
}

public function down(): void
{
    Schema::dropIfExists('etiqueta_evento');
}
```

EL NOMBRE DEL PIVOTE NO ES CASUAL

Por convención, Eloquent busca la tabla pivote uniendo los dos modelos en **singular**, **en orden alfabético** y separados por guion bajo. Como «etiqueta» va antes que «evento» (la «t» precede a la «v»), el nombre esperado es `etiqueta_evento`, no `evento_etiqueta`.

Si prefiere otro nombre, debe indicarlo en ambos lados: `belongsToMany(Etiqueta::class, 'evento_etiqueta')`. `cascadeOnDelete()` limpia el pivote cuando se borra un evento o una etiqueta.

Paso 2 — Definir belongsToMany en ambos modelos

app/Models/Evento.php Copiar

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

public function etiquetas(): BelongsToMany
{
    return $this->belongsToMany(Etiqueta::class);
}
```

app/Models/Etiqueta.php Copiar

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Etiqueta extends Model
{
    protected $table = 'etiquetas';

    protected $fillable = ['nombre', 'slug'];

    public function eventos(): BelongsToMany
    {
        return $this->belongsToMany(Evento::class);
    }
}
```

Paso 3 — Sembrar etiquetas y asignarlas a los eventos

database/seeders/EtiquetaSeeder.php Copiar

```
use App\Models\Etiqueta;
use Illuminate\Support\Str;

public function run(): void
{
    $nombres = ['Gratis', 'Aire libre', 'Música', 'Académico', 'Taller', 'Networking'];

    foreach ($nombres as $nombre) {
        Etiqueta::create(['nombre' => $nombre, 'slug' => Str::slug($nombre)]);
    }
}
```

database/seeders/EventolSeeder.php Copiar

```
use App\Models\Etiqueta;

public function run(): void
{
    $etiquetas = Etiqueta::pluck('id');

    Evento::factory()->count(20)->create()
        ->each(fn (Evento $evento) =>
            $evento->etiquetas()->attach($etiquetas->random(rand(1, 3))));
}
```

database/seeders/DatabaseSeeder.php Copiar

```
$this->call([
    CategoriaSeeder::class,
    EtiquetaSeeder::class,    // antes que los eventos: se usan en attach()
    EventolSeeder::class,
]);
```

```
terminal
$ php artisan migrate:fresh --seed
```

Paso 4 — attach, detach, sync y toggle en Tinker

```
php artisan tinker
> $evento = Evento::first();
> $evento->etiquetas()->attach([1, 2]);      // agrega las etiquetas 1 y 2
> $evento->etiquetas()->sync([2, 3]);        // deja EXACTAMENTE 2 y 3
> $evento->etiquetas()->toggle([3, 4]);      // quita la 3, agrega la 4
> $evento->etiquetas->pluck('nombre');       // colección relacionada
> $evento->etiquetas()->detach();            // quita todas
```

`sync()` es la clave para formularios: recibe la lista **final** de ids y agrega o quita lo necesario para que la relación quede exactamente así.

Paso 5 — Etiquetas en el formulario y al guardar

Entregue también las etiquetas en `create()` y `edit()`:

```
app/Http/Controllers/EventoController.php
use App\Models\Etiqueta;

public function create()
{
    $categorias = Categoria::orderBy('nombre')->get();
    $etiquetas  = Etiqueta::orderBy('nombre')->get();
    return view('eventos.create', compact('categorias', 'etiquetas'));
}
// edit(): igual, con compact('evento', 'categorias', 'etiquetas')
```

En el parcial, agregue las etiquetas al final de la sección **Datos del evento**, debajo de la fila Fecha/Lugar. Usamos casillas ocultas (`sr-only`) dentro de «chips» que cambian de color con `has-checked:`, el mismo recurso que ya usan las casillas Publicado y Destacado:

resources/views/partials/evento-form.blade.php Copiar

```
@php
    $seleccionadas = old('etiquetas', $evento?->etiquetas->pluck('id')->all() ?? []);
@endphp

<div>
    <span class="block text-sm font-medium text-slate-700">Etiquetas</span>
    <div class="mt-2 flex flex-wrap gap-2">
        @foreach ($etiquetas as $etiqueta)
            <label class="cursor-pointer rounded-full border border-slate-300 bg-white px-3
py-1 text-sm text-slate-600 transition hover:border-blue-300 has-checked:border-blue-500 has-
checked:bg-blue-50 has-checked:text-blue-700">
                <input type="checkbox" name="etiquetas[]" value="{{ $etiqueta->id }}"
class="sr-only"
                    @checked(in_array($etiqueta->id, $seleccionadas))>
                {{ $etiqueta->nombre }}
            </label>
        @endforeach
    </div>
    @error('etiquetas.*')
        <p class="mt-1.5 text-sm text-rose-600">{{ $message }}</p>
    @enderror
</div>
```

Valide el arreglo en **ambos** `validate()` y sincronice después de guardar. Note que en `store()` hay que **guardar el evento creado en una variable**, cosa que el código actual no hace:

app/Http/Controllers/EventoController.php Copiar

```
// Reglas nuevas en store() y update():
'etiquetas'   => ['nullable', 'array'],
'etiquetas.*' => ['integer', 'exists:etiquetas,id'],

// store(): antes era solo Evento::create($datos);
$evento = Evento::create($datos);
$evento->etiquetas()->sync($request->input('etiquetas', []));

// update():
$evento->update($datos);
$evento->etiquetas()->sync($request->input('etiquetas', []));
```

> **NOTA**
>
> `$datos` también trae la clave `etiquetas`, pero como no está en `$fillable` Eloquent la ignora al crear o actualizar: las etiquetas no son una columna de `eventos`, viven en el pivote y se guardan con `sync()`. El `input('etiquetas', [])` con arreglo vacío por defecto permite **quitar todas** las etiquetas al editar.

Paso 6 — Mostrar las etiquetas en el detalle

Precargue también las etiquetas en `show()` con `$evento->load(['categoria', 'etiquetas']);` y muéstrelas bajo la descripción, en la sección «Acerca del evento»:

resources/views/eventos/show.blade.php Copiar

```
@if ($evento->etiquetas->isNotEmpty())
    <div class="mt-6 flex flex-wrap gap-2">
        @foreach ($evento->etiquetas as $etiqueta)
            <span class="rounded-full bg-blue-50 px-3 py-1 text-xs font-medium text-blue-700
ring-1 ring-blue-200">
                #{{ $etiqueta->nombre }}
            </span>
        @endforeach
    </div>
@endif
```

Commits sugeridos para esta guía

terminal Copiar

```
$ git commit -m "Agrega modelo Categoria con migración y seeder"
$ git commit -m "Agrega clave foránea categoria_id a eventos"
$ git commit -m "Define relación 1:N entre Evento y Categoria"
$ git commit -m "Muestra categoría en formulario, tarjeta y detalle"
$ git commit -m "Aplica eager loading y filtro por categoría en la cartelera"
$ git commit -m "Agrega etiquetas con relación N:M"
```

> **LO QUE ACABA DE LOGRAR**
>
> Los datos de eventos-app ya están **relacionados**: cada evento tiene su categoría y sus etiquetas, con integridad garantizada por PostgreSQL, un filtro de cartelera que funciona de verdad y consultas eficientes gracias a `with()`. En la **GL09** agregaremos **autenticación** para que cada evento tenga un dueño y solo usuarios registrados puedan gestionarlos.

07 Trabajo independiente

Agregue un **organizador** a los eventos, practicando otra relación 1:N de principio a fin.

Consigna

- Cree el modelo y la tabla `organizadores` (`nombre`, `email` único) con migración, factory y seeder (al menos 5 organizadores).
- Agregue `organizador_id` a `eventos` con una migración (`constrained()` y `nullOnDelete()`), y agréguelo al `$fillable` y al `EventoFactory`.
- Defina `Evento::organizador()` (`belongsTo`) y `Organizador::eventos()` (`hasMany`).
- Agregue un selector de organizador en `partials/evento-form` con su regla `exists:organizadores,id` en `store()` y `update()`.
- Muestre el organizador como una fila más del bloque «Información» del detalle, precargándolo con `load()`.
- **Opcional (+):** agregue una columna `color` a `categorias` y haga que `x-badge` la use en lugar del `match` fijo.

Entregables

- Commits atómicos y push a `INF560-2026-02-Apellido`.

- Captura del detalle de un evento mostrando categoría, etiquetas y organizador.
- Captura de la cartelera filtrada por una categoría.

08 Checklist de verificación

- ☐ Existen el modelo `Categoria`, la tabla `categorias` y `CategoriaSeeder` con los cuatro nombres de `x-badge`.
- ☐ La tabla `eventos` tiene `categoria_id` como clave foránea a `categorias`.
- ☐ `categoria_id` está en el `$fillable` de `Evento`; el factory lo asigna y `DatabaseSeeder` siembra categorías antes que eventos.
- ☐ `Evento::categoria()` y `Categoria::eventos()` funcionan en Tinker.
- ☐ El formulario tiene un selector de categoría con `@selected` y la regla `exists` en `store()` y `update()`.
- ☐ La tarjeta y el detalle muestran la categoría con `x-badge`.
- ☐ `index()` usa `with('categoria')` y `preventLazyLoading()` no lanza excepciones.
- ☐ El filtro de la cartelera filtra por categoría y por destacados, y conserva el filtro al paginar.
- ☐ Existen `etiquetas` y el pivote `etiqueta_evento`; `belongsToMany` está definido en ambos modelos.
- ☐ `sync()` guarda las etiquetas del formulario, también al editar y al quitarlas todas.
- ☐ Trabajo independiente: relación 1:N con organizador, en GitHub.

09 Problemas frecuentes

Síntoma	Causa y solución
relation "categorias" does not exist al migrar	La migración de `categoria_id` tiene fecha anterior a `create_categorias_table`. Renombre el archivo con una fecha posterior y ejecute `migrate:fresh --seed`.
column "categoria_id" does not exist	No corrió la migración de la FK. Ejecute `php artisan migrate`.
El evento se guarda sin categoría	Falta `categoria_id` en el `$fillable` de `Evento`.
Undefined variable $categorias	La acción no envía `$categorias` a la vista. Revise `create()`, `edit()` e `index()` (el parcial de filtros también la usa).
Call to a member function ... on null	El evento no tiene categoría. Use el operador nullsafe: `$evento->categoria?->nombre`.
Attempted to lazy load [categoria] on model [App\Models\Evento]	Es `preventLazyLoading()` avisando de un N+1. Precargue con `with('categoria')`.
El filtro no cambia la lista	El `select` no tiene `name="categoria"` o `index()` no aplica el `when()`.
Al cambiar de página se pierde el filtro	Falta `->withQueryString()` después de `paginate()`.
relation "evento_etiqueta" does not exist	Eloquent busca `etiqueta_evento` (orden alfabético). Renombre la tabla o indíquela en `belongsToMany()`.
Las etiquetas no se guardan	Falta `sync()` tras guardar, `store()` no guarda el evento en `$evento`, o el `name` del input no es `etiquetas[]`.
The selected categoria id is invalid.	La regla `exists` no encontró el id. Verifique que la categoría exista y que el `select` envíe su `id`.

10 Comandos de referencia

relaciones y métodos usados en esta guía Copiar

```
# Generadores
php artisan make:model Categoria -ms          # modelo + migración + seeder
php artisan make:migration add_categoria_id_to_eventos_table

# Migraciones de relaciones (1:N y pivote)
$table->foreignId('categoria_id')->nullable()->constrained()->nullOnDelete();
$table->foreignId('evento_id')->constrained()->cascadeOnDelete();

# Definición en los modelos
return $this->belongsTo(Categoria::class);     # lado "pertenece a"
return $this->hasMany(Evento::class);          # lado "tiene muchos"
return $this->belongsToMany(Etiqueta::class);  # muchos a muchos

# Uso
$evento->categoria                             # navegar (objeto)
$categoria->eventos()->where(...)              # seguir consultando
Evento::with('categoria')->get()               # eager loading (evita N+1)
$evento->load(['categoria', 'etiquetas'])      # eager loading sobre un modelo
Evento::whereRelation('categoria', 'slug', 'deporte')   # filtrar
Categoria::withCount('eventos')->get()         # contar relacionados
$evento->etiquetas()->sync([1, 2])             # fijar la relación N:M
$evento->etiquetas()->attach($id)              # agregar
$evento->etiquetas()->detach()                 # quitar
```

11 Rúbrica de evaluación

Criterio	Pts
Modelo Categoria, migración y seeder compatibles con x-badge	10
Migración de categoria_id (constrained, nullOnDelete), factory y orden de seeders	10
Relación 1:N definida en ambos modelos, $fillable y prueba en Tinker	15
Selector de categoría en el formulario + validación exists en store y update	10
Categoría visible en la tarjeta y en el detalle	5
Eager loading con with()/load() y preventLazyLoading activo	10
Filtro de cartelera por categoría y destacados (whereRelation, withQueryString)	10
Relación N:M: tablas, belongsToMany, seeder y sync() desde el formulario	20
Trabajo independiente: relación con organizador	10
Total	**100**